



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

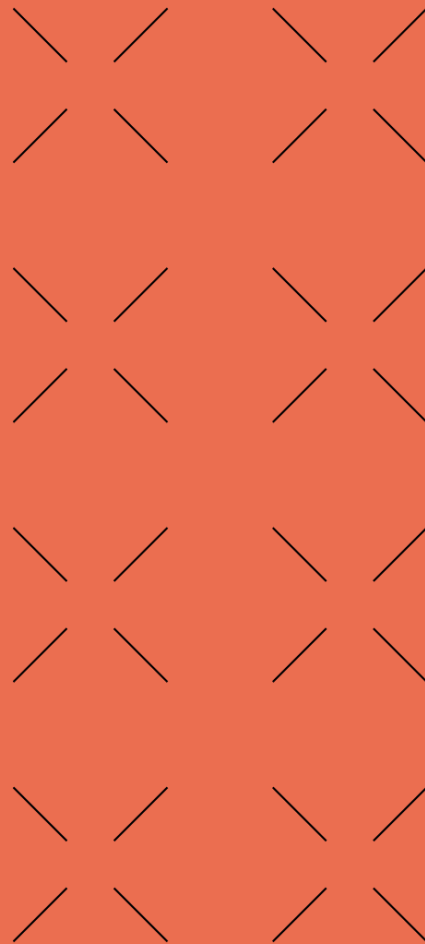
CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 07 — Algorítmica I

Bloc 1

1r CFGS DAW · Programació

Sergi García Barea



En aquesta sessió

De la recepta de cuina al codi que busca en un milió de dades

- 1 Què és un **algoritme**: la recepta, les propietats i la idea vs. el codi.
- 2 **Cerca lineal**: un a un, sense dreceres. Índex o -1 .
- 3 **Cerca binària**: obrir el diccionari per la meitat.
- 4 Traçar la binària: **esquerra, dreta, mig** — i el $+1/-1$ sagrats.
- 5 Reptes **Be the Code** i primer contacte amb $O(n)$ vs $O(\log n)$.

 El Bloc 2 porta les **ordenacions** (bombolla, inserció), **Big O** com ho escoltaràs a examen i el gimnàs de reptes.

Què és un algoritme: la recepta de l'ordinador

Un algoritme és una **seqüència finita, ordenada i sense ambigüitats** de passos que resol un problema. Com una recepta de truita de creïlles:

- 1 Pelar les creïlles i tallar-les a rodanxes.
- 2 Fregir-les amb oli abundant.
- 3 Batre els ous, barrejar-ho tot i quallar.

 **«Posa sal al gust» no val com a pas:** quant és «al gust»? Un pessic? Un grapat? Un algoritme de veritat **no deixa espai a la interpretació:** mateixa entrada → sempre mateixa eixida.

La recepta, dibuixada

La recepta: passos precisos, ordenats i deterministes



X «Posa sal al gust»

← ambigu: cadascú interpreta una quantitat diferent.
Un algoritme no deixa espai a la interpretació.



Cada pas és **precís i determinista**. El pas ambigu (sal «al gust») es queda fora: és la diferència entre una recepta i una tora.

ENTRADA

ALGORITME

EIXIDA

Les 5 propietats: finit · precís · entrada · eixida · eficaç

Les 5 propietats d'un algoritme

#	Propietat	Què exigeix
1	Finit	Ha d'acabar. Si s'executa per sempre, és un malson, no un algoritme
2	Precís	Cada pas definit sense ambigüitat: res de «al gust» ni «quan estiga llest»
3	Entrada	Rep zero o més valors d'entrada
4	Eixida	Produïx almenys un valor d'eixida
5	Eficaç	Resol el problema en temps finit i de manera correcta



Nota històrica: «algoritme» ve del matemàtic persa **Al-Juarismi** (segle IX). Segles després, els informàtics li vam robar la paraula. Som així.

La idea vs. el codi

● ● ● AlgoritmeSimple.java

```
public class AlgoritmeSimple {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 3;  
        int resultat = a + b;  
        // la idea: sumar.  
        // el codi: la materialització  
        System.out.println(resultat);  
    }  
}
```

 **No tot codi és un algoritme.** L'algoritme és la *idea*: la seqüència de passos. El codi és la seua *materialització* en un llenguatge concret.

  El mateix algoritme s'escriu igual de bé en Java, en Python o en ensamblador: són les receptes, i Java només és una de les teues cuines.

Les dues grans famílies



Cerca

Trobar un element dins d'un conjunt de dades: «el 23 és en este array?»



Ordenació

Posar les dades en un ordre determinat: «deixa este array de menor a major»

i Són les dues habilitats bàsiques de qualsevol programa que maneja dades: llistes de reproducció, bases de dades, cercadors. Sense ordenar ni buscar, la teua app és un calaix desordenat on les coses només estan «per aquí».



Be the Code: la truita ordenada

?

Estes passos de recepta estan desordenats. Ordena'ls i digues quina propietat de l'algoritme falla en l'ordre original:

- 1 Batre els ous.
- 2 Menjar la truita.
- 3 Posar sal «al gust».
- 4 Fregir les creïlles.
- 5 Pelar i tallar les creïlles.
- 6 Barrejar amb l'ou i quallar.

Ordre correcte: $5 \rightarrow 4 \rightarrow 1 \rightarrow 6 \rightarrow 2$ (el 3 sobra).

Falla la precisió: «posar sal al gust» és ambigu — cada persona interpretaria una quantitat diferent. La recepta no tindria una única manera correcta d'executar-se.

Comprovació ràpida

?

Quina és la diferència entre un algoritme i un programa?

?

Un algoritme pot rebre zero entrades?

?

Què passaria si un algoritme mai no acabara?

1. L'algoritme és la **idea**; el programa, la seua **materialització** en un llenguatge. · **2.** Sí: «imprimeix els nombres de l'1 al 10» no necessita cap entrada. · **3.** Deixaria de ser un algoritme: incompleix la propietat de ser **finit**.

Cerca lineal: la sabatilla perduda

La cerca lineal recorre l'array **element per element** fins a trobar l'objectiu o comprovar que no hi és. Com buscar la sabatilla: davall del llit, darrere de la porta, en l'armari...

CercaLinial.java

```
public static int buscar(int[] array, int objectiu) {  
    for (int i = 0; i < array.length; i++) {  
        if (array[i] == objectiu) {  
            return i; // trobat! la posició  
        }  
    }  
    return -1; // l'índex impossible: no hi és  
}
```



-1 és la convenció clàssica de «no trobat». Mai 0: la posició 0 és vàlida i diferent de «no hi és» — el clàssic «bug de l'índex zero».



Be the Code: el cercador que es perd

Cerca2.java

```
int[] dades = {3, 8, 1, 9, 5, 2};
int passos = 0;
for (int i = 0; i < dades.length; i++) {
    passos++;
    if (dades[i] == 9) {
        System.out.println(
            "Passos: " + passos);
        return i;
    }
}
// si acaba: return -1
```

Sense executar: quantes comparacions fa i què imprimix?

Compara **3, 8, 1, 9** → trobat al 4t pas.

Eixida: He necessitat 4 passos. ·

Posició: 3

El `return` talla el mètode tan bon punt troba: el bucle **no** recorre tot l'array.

Com de ràpida és la cerca lineal?

Cas	Quants passos
Millor cas — l'element és el primer	1 sol pas
Pitjor cas — és l'últim o no existix	recorres els n elements sencers

 Complexitat **$O(n)$** , lineal: 10 elements $\rightarrow$ ~10 passos; 10.000 elements $\rightarrow$ ~10.000 passos.
Creix al mateix ritme que les dades.

  **Don Tip:** la cerca lineal és com buscar en la teua nevera: si és xicoteta, tant li fa el mètode. Però amb un magatzem de 10.000 productes necessites alguna cosa millor... i en la pròxima diapositiva la trobes.

Cerca binària: obrir el diccionari per la meitat

Buscar una paraula al diccionari: no comences a la pàgina 1. Obres per la meitat, veus si la paraula va abans o després, i **descartes mitja llibre en un gest**. I repetix.

  **El requisit imprescindible: l'array ha d'estar ordenat.** Si no, el mètode no funciona — i el pitjor de tot: **no t'avisa**. No hi ha error ni excepció: simplement obtens la resposta equivocada.

 És com buscar «berenar» en un diccionari amb les paraules a l'atzar: obrir per la meitat no et servix de res.

Les dues cerques, dibuixades

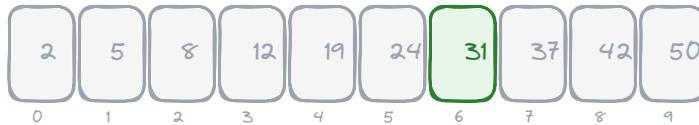
CERCA LINEAL — un a un, sense dreceres



evisa totes les places fins a trobar el 23 → 5 comparacions

Funciona amb dades desordenades

CERCA BINÀRIA — partir per la meitat



descartat

segment actiu

descartat

Exigix array ordenat · $O(\log n)$ → 1 milió → 20 passos

i **Lineal:** un a un, sense dreceres, funciona sempre · **Binària:** partir per la meitat, exigix ordre.
Amb un milió d'elements: 1.000.000 passos vs 20.

L'algoritme de la cerca binària

CercaBinaria.java



```
public static int buscar(int[] array, int objectiu) {  
    int esquerra = 0;  
    int dreta = array.length - 1;  
    // dos punters  
    while (esquerra ≤ dreta) {  
        int mig = esquerra + (dreta - esquerra) / 2;  
        if (array[mig] == objectiu) {  
            return mig;           // bingo!  
        } else if (array[mig] < objectiu) {  
            esquerra = mig + 1;    // descarta l'esquerra  
        } else {  
            dreta = mig - 1;       // descarta la dreta  
        }  
    }  
    return -1;                   // no trobat  
}
```

Traçant la cerca del 31

Array ordenat de 12 elements: {2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63}

Volta	esquerra	dreta	mig	array[mig]	Què passa?
1	0	11	5	24	$24 < 31 \rightarrow$ <code>esquerra = 6</code>
2	6	11	8	42	$42 > 31 \rightarrow$ <code>dreta = 7</code>
3	6	7	6	31	Bingo! $\rightarrow$ retorna 6



Tres comparacions. La cerca lineal n'hauria necessitat set. I amb un milió d'elements la diferència és escandalosa: 20 passos vs un milió.

Per què esquerra + (dreta - esquerra) / 2?

La fórmula anti-desbordament

```
// ⚠️ (esquerra + dreta) / 2:  
// amb arrays gegants, la suma  
// pot desbordar l'int i fer-se  
// negativa de sobte  
// ✅ la fórmula segura:  
int mig =  
    esquerra + (dreta - esquerra) / 2;
```



Si l'array té a prop de

`Integer.MAX_VALUE` elements, `esquerra`
+ `dreta` **desborda**: el resultat ja no cap en
un int.



⚠️ Este bug és tan famós que va
aparèixer en la biblioteca original de Java.
Du anys col·leccionant trofeus: Bug de
l'any, Bug de la dècada, Bug favorit del
públic...

$O(\log n)$: creix molt a poc a poc

Mida de l'array	Passos màxims de la binària
16 elements	4
32 elements	5
1.024 elements	10
1.000.000 elements	20

 **$O(\log n)$, logarítmica:** en cada pas es descarta **la meitat** del que queda. «log n» és en base 2: quantes voltes pots partir entre 2 abans d'arribar a 1.

 Torna a llegir-ho: un milió d'elements, **vint passos**. La lineal necessita un milió.



Be the Code: la traça del 8

Traca.java

```
int[] dades = {1, 4, 8, 12, 20, 33};
int esquerra = 0;
int dreta = dades.length - 1;
while (esquerra ≤ dreta) {
    int mig =
        esquerra + (dreta - esquerra) / 2;
    if (dades[mig] == 8) {
        return mig;
    }
    // ... mou esquerra o dreta
}
```

Traça la cerca del 8: esquerra, dreta i mig en cada volta.

Volta única: `esquerra=0 dreta=5 mig=2`

→ `dades[2] = 8` . **Bingo a la primera.**

Eixida: `Resultat: 2` — el millor cas de la binària: l'objectiu just al centre.

Comprovació ràpida

?

Què retorna la cerca lineal quan l'element no és en l'array?

?

Quants passos màxims necessita la binària per a 1.000.000 d'elements?

?

Què passa si fas la binària sobre un array desordenat?

1. `-1` : la senyal clàssica de «no trobat». · 2. **20 passos** — $\log_2(1.000.000) \approx 20$. · 3. Torna la resposta equivocada **sense avisar**: no hi ha error ni excepció, hi ha escombraria silenciosa.

Resum del bloc 1 en 5 frases

- 1 Un **algoritme** és una recepta finita, precisa i sense ambigüitats: l'algoritme és la **idea**, el codi la materialització.
- 2 La **cerca lineal** recorre un a un, funciona amb desordenats i torna l'índex o **-1**. Complexitat $O(n)$.
- 3 La **cerca binària** descarta la meitat en cada pas: **exigix array ordenat** o torna escombraria sense avisar.
- 4 La recepta mental: dos punters, `while (esquerra ≤ dreta)`, mig anti-desbordament i els **+1/-1 sagrats**.
- 5 **$O(\log n)$** : un milió d'elements en 20 passos. La lineal en necessita un milió.

